

# L07 – Physics-informed Neural Networks (PINN)

Foundations Lecture Series

Course Instructor

Microgrid 3-phase Security AI Project

Foundations Lecture 7

Keywords: physics in the loss, soft constraints, limit-aware learning, multi-task, project's PI-MLP

# Lecture roadmap

- 1 Where PINNs come from – the Raissi 2019 lineage
- 2 Loss design: data + physics + a knob  $\lambda$
- 3 Soft vs strict constraints, and what the project uses
- 4 Limit-aware learning – the project's specific flavor
- 5 Worked example: regularizing an MLP with safety rules
- 6 When PINNs help and when they hurt
- 7 Reading the project's PI-MLP composite loss

## Prerequisite

L05 (MLP loss design, PyTorch). Helpful: L01–L04 for the physical constraints we will use, and L06 for the GNN angle.

# The Raissi 2019 idea, in one slide

**Classical setting:** solve a partial differential equation

$$\mathcal{N}[u](x, t) = 0, \quad u(x, 0) = u_0(x), \quad u(x, t)|_{\partial\Omega} = g$$

by training a neural net  $u_\theta(x, t)$  with the loss

$$\mathcal{L} = \underbrace{\frac{1}{N_d} \sum_i \|u_\theta(x_i, t_i) - u_i\|^2}_{\text{data fit}} + \lambda \cdot \underbrace{\frac{1}{N_c} \sum_j \|\mathcal{N}[u_\theta](x_j, t_j)\|^2}_{\text{physics residual at collocation points}}$$

- Data points  $x_i, t_i$  where we measure  $u$  – typically few.
- Collocation points  $x_j, t_j$  where the PDE *should* hold – can be cheap to sample on a grid.
- Net learns to fit data *and* respect the equation.

## Why this is powerful

The physics term gives the model a strong inductive prior. It often extrapolates outside the data region in a way a pure data-fit net cannot.

# Three reasons to put physics in the loss

## 1. Sample efficiency

- PDE residual is a “free supervisor” anywhere we can evaluate it.
- Useful when labeled data is expensive (e.g. instrumentation, expert labels).

## 2. Extrapolation

- A plain net is reliable only where it has data.
- Physics terms constrain the model in unmeasured regions.

## 3. Trustworthiness

- Hard for a non-expert to audit a black-box prediction.
- Easier to defend a model that “never violates Ohm’s law” or “always respects voltage limits”.

### For safety screening

Reasons 1 and 3 dominate. The project has only 77 labeled samples (data is expensive) and is for a safety-critical task (auditability matters). Both push toward physics-informed methods.

# The general data + physics decomposition

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda \cdot \mathcal{L}_{\text{phys}}$$

## Data loss

- BCE for classification.
- MSE for regression.
- Cross-entropy for multi-class.
- Standard ML, evaluated on labeled training samples.

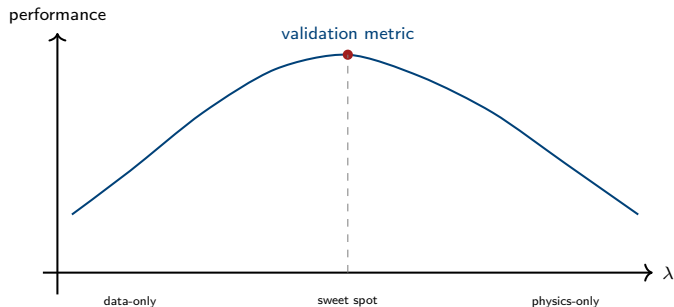
## Physics loss

- PDE residual (Raissi-style).
- Engineering limit violation (project-style).
- Conservation law residual.
- Symmetry / invariance penalty.
- Evaluated on *any* points where physics is defined.

## Where physics points come from

For PDE problems: sample uniformly on the domain. For our project: random scenario–contingency pairs, or grid points in feature space. They do *not* need labels; the physics rule provides the supervisory signal.

# The penalty coefficient $\lambda$



## Symptoms of bad $\lambda$

- Too small: physics has no effect; same as plain MLP.
- Too large: model ignores data, predicts whatever satisfies physics.
- Just right: physics regularizes without dominating.

## Practical approach

- Start at  $\lambda = 1$ .
- Sweep over  $\{0.01, 0.1, 1, 10, 100\}$  on validation.
- Pick the  $\lambda$  that maximizes the target metric (recall, AUPRC, etc.).

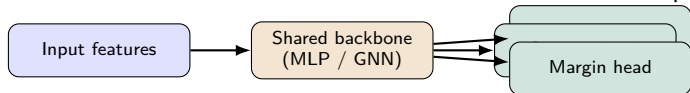
# Multi-task framing

Many real PINNs are not just one data term + one physics term – they are several losses summed:

$$\mathcal{L}_{\text{total}} = \lambda_1 \mathcal{L}_{\text{cls}} + \lambda_2 \mathcal{L}_{\text{sev}} + \lambda_3 \mathcal{L}_{\text{limit}} + \lambda_4 \mathcal{L}_{\text{rank}} + \dots$$

Each term enforces a different aspect of the desired behavior. The architecture often has multiple

output heads sharing a backbone.



This is exactly the project's Week 6 PI-MLP

The shared layers extract a common representation. Each head outputs a different quantity. Each loss term penalizes a different mistake. The  $\lambda_k$  values are tuned on validation.

# Soft constraints (the usual PINN)

A soft constraint is just a penalty term:

$$\mathcal{L}_{\text{phys}} = \frac{1}{N_c} \sum_j \| \text{constraint violation at } (x_j) \|^2$$

**Example (voltage limit):** we want the model to predict “unsafe” whenever  $|V_a| < 0.95$ . Define the penalty:

$$\mathcal{L}_{\text{V-limit}} = \frac{1}{N_c} \sum_j [\mathbb{I}(V_j < 0.95) \cdot \max(0, 0.5 - \hat{y}_\theta(x_j))^2]$$

Zero when the model already predicts  $\hat{y} \geq 0.5$  (unsafe) on those points. Quadratic and positive otherwise.

## Properties

- Easy to implement: just add a term to the loss.
- Gradient-friendly: backprop works as usual.
- Model *can* still violate the constraint – the penalty just makes it costly.



# Strict constraints (projection or architecture)

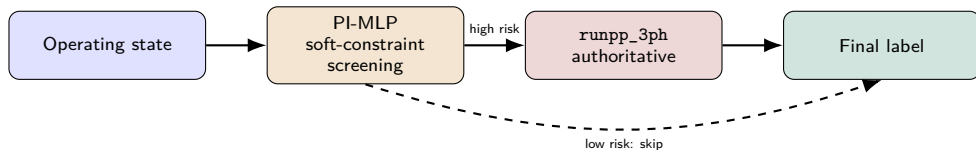
A strict constraint cannot be violated:

- **Projection:** after the model's forward pass, project the prediction onto the feasible set. E.g., for power flow, clamp predicted voltages to a known band.
- **Architecture:** design the network so its outputs structurally satisfy the constraint. E.g., output magnitudes via a softplus to guarantee positivity; output a distribution via softmax to guarantee summation to 1.
- **Lagrangian / augmented:** explicit equality constraints with Lagrange multipliers, updated during training.

## Trade-offs

Strict constraints are necessary when violations are catastrophic and there's no downstream safety net. They are harder to design and may slow training. **For a screening model with a power-flow solver fallback, soft constraints are usually enough.**

# Choosing for safety screening: soft is fine



## Why soft constraints are sufficient here

The model is a *pre-screener*. Anything it flags as suspicious is re-checked by the exact power-flow solver. So a small violation of the soft constraint at inference time does no harm – the solver catches it. Strict constraints would matter only if we removed the solver fallback.

## What changes for a deployed safety controller

If the model directly actuates an inverter or relay, we want hard constraints, conformal prediction, or formal verification. That's out of scope for screening but on the research roadmap.

# What “physics-informed” means in this project

The microgrid project does *not* encode the full AC power-flow residual into the loss. It builds five nonnegative component-margin targets from simulator outputs, then adds supervised auxiliary and

| consistency losses. | Target / term           | Quantity                            | Definition or loss                   |
|---------------------|-------------------------|-------------------------------------|--------------------------------------|
|                     | $m_{V,low}, m_{V,high}$ | minimum/maximum phase voltage       | normalized positive-part margins     |
|                     | $m_I$                   | maximum phase loading vs 100%       | $[L_{max}/100 - 1]_+$                |
|                     | $m_U$                   | sequence VUF vs 2%                  | $[U_{max}/2 - 1]_+$ (VUF in percent) |
|                     | $m_S$                   | service loss / no-slack flags       | maximum topology-risk flag           |
|                     | $\mathcal{L}_{cls}$     | probability head $\hat{p}$ vs label | weighted BCE                         |
|                     | $\mathcal{L}_{comp}$    | predicted vs target margins         | SmoothL1 on $\log(1 + m)$            |

## Honest framing

This is *limit-aware* learning rather than strict Raissi-PINN. The same name (“physics-informed”) has stretched to cover both. State which flavor you mean in your write-up so readers know what to expect.

# Severity regression as physics-informed

Instead of only predicting a binary label, also predict a *continuous severity score*:

$$s_{PI} = \max\{s_{raw}, m_{V,low}, m_{V,high}, m_I, m_U, m_S\}$$

- Larger  $s$  means worse violation;  $s = 0$  means safe.
- The severity head learns  $s_{PI}$  with SmoothL1 loss after a  $\log(1 + s)$  transform.
- The classification head and severity head share the backbone, so the two tasks help each other.

## Why this helps

A single binary label compresses a lot of information into one bit. Severity gives the network a much richer supervisory signal – specifically a *continuous physical quantity*. The classification head benefits from learning the severity geometry alongside.

# Ranking loss for top- $k$ screening

For pre-screening, the *order* of risk scores matters more than absolute probabilities. We may not have well-calibrated probabilities, but we want unsafe scenarios to be at the top of the queue.

$$\mathcal{L}_{\text{rank}} = \frac{1}{|\mathcal{P}|} \sum_{(i,j) \in \mathcal{P}} \max(0, m - (\hat{y}_i - \hat{y}_j))$$

In Week 6, pairs are ordered by continuous target severity:  $(i,j) \in \mathcal{P}$  when  $s_i > s_j$ . The model ranks the predicted severity head with margin  $m = 0.05$ .

- Pairwise hinge penalizes a more severe case receiving a lower predicted severity.
- It pushes the higher-severity score above the lower one by at least margin  $m$ .
- Improves top- $k$  hit rate, which is the project's actual operational metric.

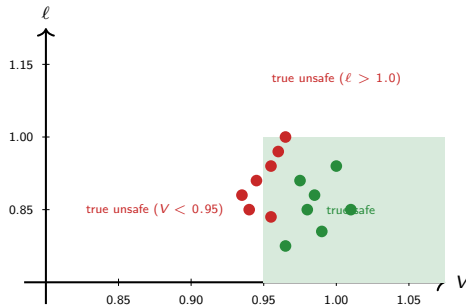
## What kind of prior is this?

The ranking term is task-aware rather than a physical residual. It is included because the deployment objective is contingency prioritization; the physical content enters through the severity and component-margin targets.

## Toy problem: predict “safe” from $V$ and current loading $\ell$

**Ground-truth rule.** A state  $(V, \ell)$  is **unsafe** iff  $V < 0.95$  or  $\ell > 1.0$ . Otherwise safe.

**Training data.** Only 30 labeled scenarios, clustered near the boundary at  $(V, \ell) \in [0.93, 1.02] \times [0.85, 1.05]$ .



*No labels in the far corners.*

### The challenge

Plain MLP fits the local data well. But there are no training points at  $V = 0.85$  or  $\ell = 1.3$ , so its prediction in those regions is arbitrary. We don't trust it for safety screening.

# Adding a limit-aware physics penalty

**Physics rule.** At any feature-space point  $(V, \ell)$ :

- if  $V < 0.95$  or  $\ell > 1.0$ , the prediction should be  $\hat{y} \geq 0.5$  (unsafe).
- if  $V \geq 0.95$  and  $\ell \leq 1.0$ , the prediction should be  $\hat{y} < 0.5$  (safe).

**Physics loss.** Sample  $M = 1000$  random points  $(V_j, \ell_j)$  uniformly over  $[0.85, 1.10] \times [0.60, 1.30]$  – no labels needed – and add:

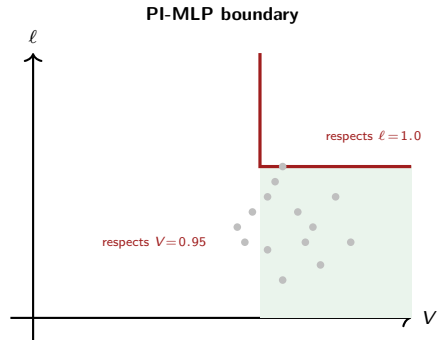
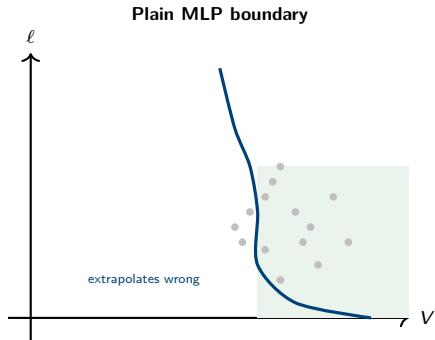
$$\mathcal{L}_{\text{phys}} = \frac{1}{M} \sum_j \left[ \mathbb{I}(V_j < 0.95 \text{ or } \ell_j > 1.0) \cdot \max(0, 0.5 - \hat{y}_\theta(V_j, \ell_j))^2 \right. \\ \left. + \mathbb{I}(V_j \geq 0.95 \text{ and } \ell_j \leq 1.0) \cdot \max(0, \hat{y}_\theta(V_j, \ell_j) - 0.5)^2 \right]$$

**Total loss.**  $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{BCE on 30 labeled pts}} + \lambda \cdot \mathcal{L}_{\text{phys}}$ , with  $\lambda = 1.0$ .

## Where the “physics” is

The rule (“V or current outside limits  $\Rightarrow$  unsafe”) comes from engineering knowledge, not from the 30 labels. The penalty term injects this knowledge anywhere in the feature space, even where data is absent.

# Plain MLP vs PI-MLP – what changes



## What the picture says

The plain MLP draws a curved boundary that fits the training points but bends in unexpected directions outside the data cluster. The PI-MLP draws a clean L-shape that matches the engineering rule everywhere – even at  $V = 0.85$  where no training data lives.



## In code – adding a physics loss to a PyTorch training loop

```
import torch, torch.nn as nn

# Plain MLP as in L05
model = nn.Sequential(nn.Linear(2, 32), nn.ReLU(),
                      nn.Linear(32, 32), nn.ReLU(),
                      nn.Linear(32, 1), nn.Sigmoid())
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
bce = nn.BCELoss()

def physics_loss(x, y_hat):
    V, I = x[:, 0], x[:, 1]
    should_be_unsafe = ((V < 0.95) | (I > 1.0)).float().unsqueeze(1)
    should_be_safe = 1.0 - should_be_unsafe
    pen_unsafe = torch.relu(0.5 - y_hat) ** 2 * should_be_unsafe
    pen_safe = torch.relu(y_hat - 0.5) ** 2 * should_be_safe
    return (pen_unsafe + pen_safe).mean()

for epoch in range(200):
    # data loss on 30 labeled points (X_tr, y_tr)
    loss_data = bce(model(X_tr), y_tr)
    # physics loss on 1000 random unlabeled points (X_phys)
    loss_phys = physics_loss(X_phys, model(X_phys))
    loss = loss_data + 1.0 * loss_phys
    opt.zero_grad(); loss.backward(); opt.step()
```

# When PINN helps

- **Sparse data.** Few labels but plenty of room to evaluate the physics. Most successful PINN applications fall here.
- **Known physics.** The constraint is correct and easy to evaluate. Voltage limits and Ohm's law qualify; "a wise human would say this is unsafe" does not.
- **Extrapolation matters.** The model will be queried outside the training distribution. Physics provides a fallback.
- **Safety-critical applications.** You need to defend the model's behavior to regulators, operators, or reviewers.

## Why the project qualifies for all four

77 labeled samples, well-defined voltage / current / VUF limits, N-1 contingency analysis is inherently out-of-distribution (different from the base case), and the application is grid security.

## When PINN does not help (or hurts)

- 1  **Plenty of data, well-tuned baseline.** If a plain MLP already gets 99% recall on a labeled dataset of millions, the physics term is noise. Common finding in large-scale tabular benchmarks.
- 2  **Wrong physics.** If the constraint is a stale or approximate rule, it can pull the model away from the true distribution. Audit the constraint before trusting it.
- 3  **Overweighted  $\lambda$ .** The model fits physics perfectly and ignores data. Showing up as “training BCE never drops” in your logs.
- 4  **Optimization landscape conflicts.** Two competing terms with mismatched gradient magnitudes can stall training. Krishnapriyan et al. 2021 documents many failure modes.
- 5  **Treating it as magic.** PINNs need just as much hyperparameter tuning, just as much validation discipline, as plain MLPs.

## The six losses in Week 6

$$\mathcal{L} = \mathcal{L}_{cls} + \lambda_s \mathcal{L}_{sev} + \lambda_c \mathcal{L}_{comp} + \lambda_{cons} \mathcal{L}_{cons} + \lambda_p \mathcal{L}_{prob-sev} + \lambda_r \mathcal{L}_{rank}$$

| Term                     | Role                            | Form                               |
|--------------------------|---------------------------------|------------------------------------|
| $\mathcal{L}_{cls}$      | main classifier                 | weighted BCE                       |
| $\mathcal{L}_{sev}$      | continuous severity signal      | log-SmoothL1                       |
| $\mathcal{L}_{comp}$     | five component-margin targets   | log-SmoothL1                       |
| $\mathcal{L}_{cons}$     | severity covers worst component | $[\max_k \hat{m}_k - \hat{s}]_+^2$ |
| $\mathcal{L}_{prob-sev}$ | probability/severity agreement  | MSE between mapped scores          |
| $\mathcal{L}_{rank}$     | top- $k$ screening order        | pairwise hinge                     |

### Default weights used in the notebook

$\lambda_s = 0.50$ ,  $\lambda_c = 0.25$ ,  $\lambda_{cons} = 0.20$ ,  $\lambda_p = 0.10$ , and  $\lambda_r = 0.20$ . These are teaching defaults; the notebook reports an ablation, not an exhaustive hyperparameter claim.

# Reading the ablation table

The project's Week 6 ablation peels each term off and re-trains:

| Model variant                                   | Recall       | Precision    | PF calls saved | Missed violations |
|-------------------------------------------------|--------------|--------------|----------------|-------------------|
| BaselineMLP (only $\mathcal{L}_{\text{cls}}$ )  | 1.000        | 0.857        | 0.300          | 0                 |
| PI-MLP no-rank ( $-\mathcal{L}_{\text{rank}}$ ) | 1.000        | 0.923        | 0.350          | 0                 |
| <b>PI-MLP full</b> (all 6 terms)                | <b>1.000</b> | <b>1.000</b> | <b>0.400</b>   | <b>0</b>          |

## Reading the trend

- Each added term improves precision and PF-calls-saved.
- Recall stays at 1.0 throughout (no missed violations).
- The ranking term contributes the final precision lift.

## Honest caveat

77 samples. The ablation tells a clean story *on this dataset*; scaling to hundreds of feeders is needed to claim the ranking is operationally robust.

# Homework

## Required

- 1 Reproduce the toy  $(V, \ell)$  example. Train a plain MLP on 30 labeled points, then a PI-MLP with the limit-aware physics loss. Plot decision boundaries.
- 2 Sweep  $\lambda \in \{0, 0.01, 0.1, 1.0, 10.0, 100.0\}$ . Report the validation accuracy at each. Where does the PI-MLP start to ignore the labels?
- 3 Make the training data even sparser (10 points). Does the gap between plain MLP and PI-MLP widen?
- 4 Replace the hard rule " $V < 0.95 \Rightarrow \text{unsafe}$ " with a soft rule that has a fuzzy margin. Re-derive the penalty. Implement and compare.

## Optional

- 1 Add a third feature (VUF) and a third limit (2%). Train. How does the loss decomposition change?
- 2 Replace the MLP backbone with the GCN from L06. Implement the limit penalty per node. Does the GNN benefit?

# Exit checklist

- 1 ✓ I can write the generic PINN loss  $\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda \mathcal{L}_{\text{phys}}$  from memory.
- 2 ✓ I can name three reasons to put physics in the loss.
- 3 ✓ I can distinguish a soft constraint from a strict / projected one and pick the right tool.
- 4 ✓ I can list the six Week 6 loss terms and distinguish physical targets from task-aware regularizers.
- 5 ✓ I can implement a limit-aware physics loss in PyTorch in  $\sim 10$  lines.
- 6 ✓ I can name two scenarios where PINNs help and two where they hurt.
- 7 ✓ I can read the Week 6 ablation table and explain which loss term contributes which gain.

# References

- **Raissi, Perdikaris, Karniadakis, 2019.** *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, JCP. The canonical reference.
- **Karniadakis et al., 2021.** *Physics-informed machine learning*, Nature Reviews Physics. Comprehensive survey.
- **Krishnapriyan et al., 2021.** *Characterizing possible failure modes in physics-informed neural networks*, NeurIPS. Required reading on PINN pathologies.
- **Misyris, Venzke, Chatzivasileiadis, 2020.** *Physics-informed neural networks for power systems*, IEEE PES General Meeting. Power-system application.
- **Stiasny, Misyris, Chatzivasileiadis, 2021.** *Physics-informed neural networks for non-linear system identification: applications to power systems*, IEEE Madrid PowerTech.
- **Donon et al., 2020.** *Graph Neural Solver for Power Systems*, IJCNN. GNN + physics.